NETS 2120 - FINAL PROJECT REPORT- INSTAGRAM-CLONE-GROUP

Team members: Shreya Mukunthan, Stefan Matic, Faiyaz Hasan, Jimena Olivares Rivera

Introduction: This project involved building InstaLite, a full-stack, Instagram-like social media platform that supports user posts, comments, hashtags, real-time updates via Kafka, and personalized feed ranking using the adsorption algorithm in Apache Spark. It integrates distributed systems, vector search, cloud storage, chatbot, Chroma image uploads, web sockets, and a React frontend to create a scalable, dynamic user experience.

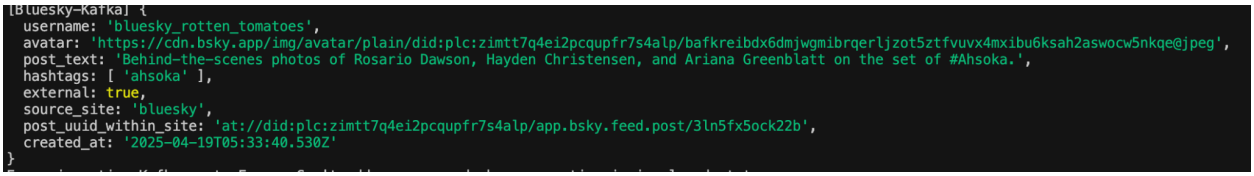
Kafka Integration

We subscribed to two Kafka topics:

1. **Bluesky-Kafka**: Contains streaming posts from a movie-focused social network. These are parsed, validated, and inserted into our posts table.
2. **FederatedPosts**: Contains inter-team posts. To ensure consistency, we normalized usernames and created pseudo-user entries in the users table for any foreign users. These pseudo-users are treated as valid nodes in the ranking graph.

In order to do this, we had to create both a Kafka consumer and a Kafka producer (mainly for testing purposes). The Kafka consumer was implemented in App.js, while the producer was in a Kafka directory in the server directory.

The Kafka producer created a dummy post from federated posts. The purpose of this file is to check if the database is able to differentiate between posts made on Instalite versus ones that are external. Additionally, we want to make sure duplicate posts are not added into the DB and that the consumer only reads in new posts using variable fromBeginning = false. Here is a read from the consumer.



Finally, we utilized a Kafka_db file to help add posts that were read in from external sources into the database.

Kafka integration challenges:

- **Schema mismatch**: We implemented schema validation to avoid runtime ingestion errors. We specifically faced this with JSON errors, as FederatedPosts do not come with hashtags while Bluesky-Kafka posts do.
- **Backpressure**: Our consumer buffers were overflowing due to high write rates. In fact, there were several RAM issues on our computers. Fixed this through utilizing wrappers.
- **Identifying external posts from internal posts**: edited names of users such that all external posts were made by external users.
- **Prevent Duplicates:** Several duplicate posts are read in, handled this using checks before adding anything into the posts and users databases against the existing data. Also utilized fromBeginning variable.

Adsorption Algorithm and Spark Job

The Adsorption algorithm computes relevance scores of all posts for each user in the social graph. Each node (user, post, hashtag) gets a label vector representing probability mass for each user. This mass is propagated iteratively via weighted edges, and reset partially to the user source to ensure convergence (alpha jumps).

Graph Construction: We constructed the graph as mentioned in the instructions.

```
DEBUG: Updated ranks for 9 records
Feed ranking job completed successfully!
```

After label convergence (typically 15 iterations), we filter the pX nodes and extract label scores. Each (user, post, score) triplet is inserted into ranked_feed, and a separate SQL update computes rank by sorting descending score per user.

Design Decisions

- **Alpha blending**: We experimented with alpha = 0.85 as our jump probability. Lower values caused diffusion of label vectors and poor personalization. This value balanced convergence and personalization well.
- **User node prefixes**: We explicitly prefixed IDs with u, p, h to prevent collisions in the Spark job. This simplification allowed cleaner filtering and analysis.
- **Feed source fallback**: When no ranked posts were available (due to low activity), the frontend gracefully fell back to a chronological list of the user's own posts.

Challenges and fixes:

- **Users not seeing their own posts**: Initially, we did not reintroduce mass to self-labeled users after propagation, resulting in label diffusion. We fixed this by introducing a restartLabels map that re-injected mass with (1 - alpha) at each iteration.
- **Poor propagation on federated posts**: Many Kafka users didn't exist in our users table. These nodes lacked outgoing edges, resulting in dead ends. We resolved this by adding synthetic users for all authors in Kafka topics.
- **Normalization loss**: Some nodes had no outgoing edges (e.g., isolated hashtags), causing label mass to vanish. We added normalization safeguards post-aggregation to preserve distributions.
- **Running out of RAM**: This ended up being an issue with JSON parsing, but we ran out of RAM several times when running the algorithm. To help with this, we changed the number of iterations of adsorption.
- **JAR Packaging Errors**: We encountered silent failures with malformed paths when creating the Spark job JAR. The fix involved explicitly navigating to out/ and targeting .class files.

Vector Image Upload and Frontend Integration:

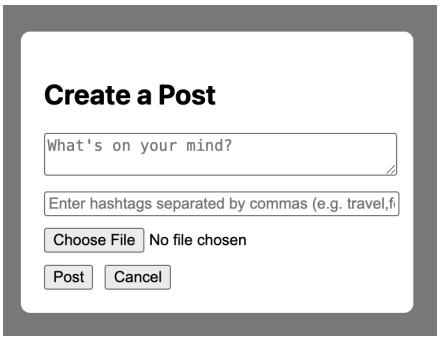
For the InstaLite project, the system implemented and refined user-facing features related to profile creation, authentication, image upload, and persistent user personalization. A major functionality involved a profile image selection workflow that was triggered after a user registered and logged in for the first time. In this flow, the user was guided to a dedicated profile photo selection view, where they could upload an image. Once uploaded, the image was processed by the backend, which generated vector embeddings and compared them against a database of actor face embeddings. The backend returned the uploaded image along with the five closest actor matches, which were then rendered in an interactive grid layout.

The system allowed users to click on any of the six image options to finalize their profile photo. Upon selection, the image URL was stored in localStorage, ensuring persistence across sessions. If the user selected an actor match, the system sent a POST request to the backend to link that actor ID with the user's account. This selected image was then displayed across the application, including in the sidebar and the user's profile page. To ensure consistent display even after logout and subsequent logins, logic was implemented to retrieve and display the saved profile image based on stored session data or a fetch call to the backend, falling back to a placeholder only if necessary.

Additionally, the system included a “Change Profile Photo” button within the UserPage view, enabling users to revisit the photo selection interface after account creation. This involved establishing a new route and ensuring that image reselection did not conflict with existing profile logic.

On the backend, AWS S3 was used for image storage. The system handled image uploads, generated signed URLs for retrieval, and ensured that the returned links were compatible with browser rendering. Issues involving broken image links were resolved by checking file upload integrity, verifying access permissions, and properly formatting the response URLs.

The development team also encountered issues with ChromaDB, particularly involving disk I/O errors and deprecated flag usage when configuring persistent storage. These problems were resolved by directly launching the Chroma service using the --path flag and setting up the data directory to avoid connection failures during vector retriever initialization. The backend was then successfully reconfigured to resume retrieval and collection creation.

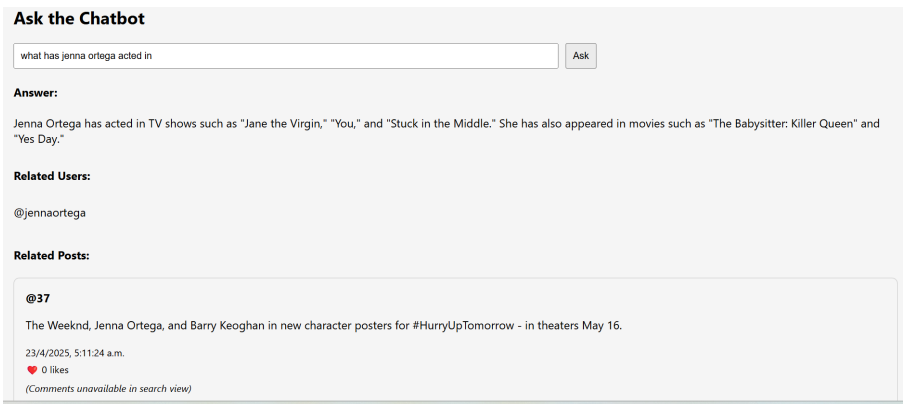


While this subsystem handled profile-specific logic and image persistence, the broader development team worked on complementary modules such as post creation, real-time chat, the activity feed, and the social graph. API routes and data schema were jointly designed to support frontend-backend communication, and Git was used to integrate individual contributions. Throughout the development process, close coordination ensured that each component—from image selection to actor linking to data persistence—integrated seamlessly into the overall user experience.

Chatbot, Embeddings, and Backend Infrastructure

InstaLite combines a robust backend infrastructure with a vector-search-powered chatbot that allows users to search for content using natural language. These systems work together to support persistent user activity while enabling meaningful discovery of posts, users, and external movie data.

The backend is built on Amazon RDS using MySQL and stores all core data for the application, including users, posts, comments, friendships, hashtags, and chat messages. The database schema is designed to balance structure and flexibility. The users table includes securely hashed passwords, editable profile fields, and a JSON array to store user interests as hashtags. The posts table tracks text captions, optional S3-hosted images, timestamps, and hashtags. Comments are linked to both posts and users, with support for pagination to manage large threads. For chat functionality, the system uses chat_sessions and chat_messages tables that store group membership and message history. All data is accessed through RESTful API endpoints, which power the corresponding views in the React frontend, including login, the feed, profile updates, and search.



On the frontend, the chatbot lives on a dedicated search page. When a user navigates to this page, the system

generates fresh vector embeddings for all users and posts currently in the database. These embeddings are then inserted into a temporary ChromaDB collection that reflects the latest activity on the platform. This approach avoids stale results and ensures that searches always return the most relevant and recent content. By contrast, external content such as actor blurbs and movie data does not change. These are embedded once at the start of the application and stored in long-term ChromaDB collections.

The chatbot uses LangChain's Retrieval-Augmented Generation (RAG) pipeline to respond to user questions. Each query is compared against three sources: actor descriptions from Homework 4, movie data from the Kabble feed, and real-time platform data like posts and profiles. ChromaDB collections are used to find the most semantically relevant documents in each category. The chatbot then builds a prompt using these documents and sends it to OpenAI's language model to generate a final response. Results can include actor biographies, matched posts, or suggested user profiles, each of which links back to live content in the app.

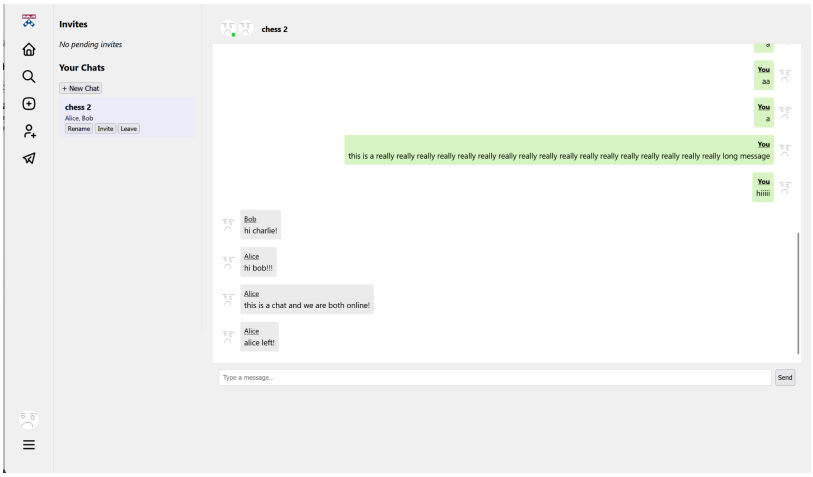
This system meets several extra credit goals from the project specification. First, it enables natural language search over both static and dynamic content using retrieval and generation. Second, it supports actor-profile matching by comparing the user's selfie to precomputed actor embeddings and showing the top five closest matches. When a user selects a match, a status update is posted to reflect the change. Third, the system embeds live post and user data during each search session to reflect real-time changes in the platform.

By designing separate workflows for static and dynamic embeddings, using ChromaDB to support fast similarity search, and building a seamless frontend interface to surface results, the system creates a powerful and intuitive way for users to explore the platform. These tools not only enrich the user experience but also provide a foundation for scalable, intelligent content discovery in future extensions of the project.

Technical Overview of WebSocket and Chat Functionality

For the InstaLite project, real-time chat technology was implemented using WebSocket, specifically leveraging socket.io to handle direct, two-way communication between the client and server. Socket.io's event-driven design allowed the chat interface to update almost instantly, ensuring quick responses and a seamless chatting experience.

Persistent chat sessions were a key design feature, allowing users to reconnect and seamlessly pick up previous conversations or start new sessions with existing contacts. Chat histories were stored securely on AWS RDS, and a specialized SQL schema was designed and optimized for fast retrieval, ensuring smooth performance even as chat history grew.



Support for both individual chats and group conversations were created. To prevent confusion or duplication, logic to keep each chat unique was added, even when the same users participated in different conversations simultaneously. Real-time notifications were implemented for chat invitations, giving users immediate control to accept or decline, with UI updates handled instantly via socket.io events.

Ensuring real-time accuracy in user status was crucial. Socket.io events were used to broadcast user connection and disconnection status, so the avatars and names of online users were always correctly displayed and updated immediately for everyone. When users disconnected, their presence instantly vanished from the chat interface across all participants, maintaining clarity and accuracy.

Users were allowed to rename chat sessions dynamically. These renaming events were instantly propagated through WebSockets, so all chat participants saw the new name right away, including users who were not in the chat but were invited. This significantly improved clarity and user experience within group chats.

Additionally, a safe and automatic deletion system for chats. If a chat became empty it was automatically removed from the database to prevent unnecessary clutter and keep the system clean.

Message ordering was another priority; by using server-generated timestamps, all messages appeared in the correct chronological order consistently across every client.

Security was deeply integrated into the design. Robust user authentication was used to prevent unauthorized access or impersonation. Moreover, scalability was optimized by fine-tuning socket.io configurations, allowing the system to handle increased user loads efficiently, supported by robust AWS infrastructure.

Overall Lessons Learned:

1. Distributed Systems Require Careful Coordination
    - Orchestrating communication between independent components like Kafka, Spark, and MySQL required deep understanding of how data flows across systems. I learned how easily things break if even one piece—like Kafka not consuming messages or Spark output not persisting to the database—fails silently. Logging and observability were essential.
2. Real-Time + Batch Architectures Need Smart Boundaries
    - Building both real-time (Kafka-based posts/comments) and batch (hourly Spark ranking) components taught me to think critically about which operations are latency-sensitive and which benefit from periodic aggregation.
3. Data Model Design Affects Algorithm Effectiveness
    - I saw firsthand how adding Bluesky and FederatedPosts users into the main users table significantly improved label propagation during adsorption. Good schema design isn't just a backend decision—it directly affects ML and ranking quality.
4. Restart Probability in Graph Algorithms Helps Reduce Bias
    - Incorporating α (alpha) for random jumps helped balance personalization with diversity in the feed. This taught me the importance of avoiding echo chambers in recommendation systems and using mathematical tools like teleportation to smooth results.
5. SQL is Fragile When Dynamically Composed
    - Debugging SQL like the ORDER BY rank ASC syntax error reinforced the importance of writing and testing raw queries carefully, especially when ranking logic and schema fields get modified in parallel.
6. Frontend and Backend Must Be Built with APIs in Mind
    - Designing the ranking system was not enough—I had to also expose REST endpoints and SQL joins that allowed the frontend feed to query relevant data efficiently. Building full-stack helped me develop empathy across layers.
7. Testing Kafka Locally Doesn't Guarantee It'll Work Remotely
    - I underestimated the challenges of testing Kafka in a containerized AWS setup. Port forwarding, producer/consumer isolation, and encoding mismatches (especially with federated post formats) caused non-trivial bugs that didn't appear locally.
8. Every Feature Affects Multiple Components
    - Adding "likes" impacted ranking, UI, and the database schema. This forced me to plan changes holistically and communicate changes clearly to teammates.